# **LangChain** — Overview & Architecture

## Section 00: Why LangChain Exists and How It Fits Together

LangChain v1.2.x Study Guide

March 2026

# What You Will Learn

**Section Objectives**

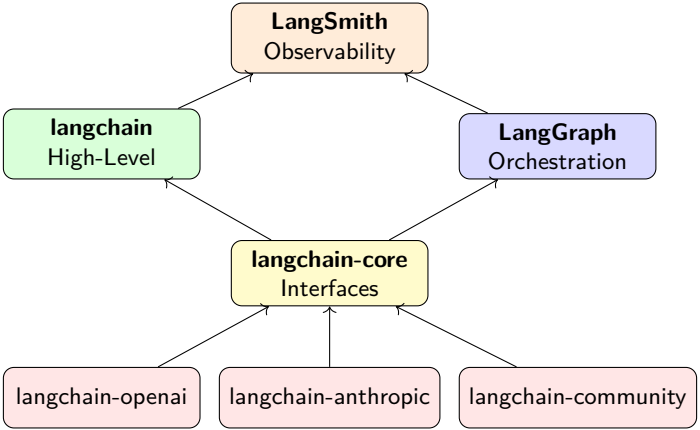
By the end of this section, you will understand:

- What problem LangChain solves and why it exists
- The LangChain ecosystem: LangChain, LangGraph, LangSmith
- The package architecture and how modules relate
- How to install and configure LangChain
- Where LangChain fits relative to alternatives

**Prerequisites:** Basic Python knowledge. No prior LangChain experience needed.

[Docs: langchain.com/docs]

# Mental Model: The LangChain Universe




[Docs: docs.langchain.com/oss/python/concepts]

# The Problem: Why LangChain?

**Building LLM applications without a framework means:**

- Writing custom glue code for every LLM provider
- Manually managing prompt templates, output parsing, error handling
- Reinventing conversation memory, tool calling, retrieval patterns
- No standardized way to compose, test, or observe your pipelines

**LangChain's Value Proposition**

LangChain provides **standardized interfaces** for LLM components and **composable primitives** for chaining them together — so you can swap providers, add tools, and build agents without rewriting your application.

# What LangChain Is. . .

**LangChain IS**

- A framework for composing LLM calls
- An abstraction layer over providers
- A toolkit for agents, RAG, tools
- An orchestration platform (via LangGraph)
- Provider-agnostic by design

**LangChain is NOT**

- An LLM itself
- A hosted AI service
- A UI framework
- Required for all LLM apps
- A replacement for understanding LLMs

# When to Use LangChain

**Good Fit**

Use LangChain when you need **multi-step LLM workflows**, **tool calling**, **retrieval**, or **agents**. It shines when you want provider flexibility and production observability.

**May Be Overkill**

For a single LLM API call with no chaining, the raw provider SDK (e.g., `openai` or `anthropic`) may be simpler and more direct.

# The Ecosystem at a Glance

| Component | Purpose | When You Need It |
|---|---|---|
| `langchain-core` | Base interfaces (Runnables, Messages, Tools) | Always (auto-installed) |
| `langchain` | High-level chains, agents, retrieval | Most applications |
| `langgraph` | Graph-based agent orchestration | Complex agent workflows |
| `langsmith` | Tracing, evaluation, monitoring | Testing & production |
| `langchain-openai` | OpenAI integration | If using OpenAI models |
| `langchain-anthropic` | Anthropic integration | If using Claude models |
| `langchain-community` | 700+ third-party integrations | As needed |

[Docs: docs.langchain.com/oss/python/concepts/architecture]

# Installation

## Basic Installation

```
# Core + high-level features
pip install langchain

# Add provider packages as needed
pip install langchain-openai        # OpenAI models
pip install langchain-anthropic     # Claude models
pip install langchain-google-genai  # Gemini models

# Optional: orchestration & observability
pip install langgraph
pip install langsmith
```

## Version Note

LangChain v1.2.x is the current stable release. The legacy `langchain-classic` package is **not recommended** for new projects.

# Environment Setup

## API Key Configuration

```
# Option 1: Environment variables (recommended)
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
export LANGSMITH_API_KEY="lsv2_..."  # optional

# Option 2: Pass directly in code
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(api_key="sk-...")
```

## Best Practice

Use environment variables or a `.env` file with `python-dotenv`. Never hardcode API keys in source code.

# Your First LangChain Program

## Hello LangChain

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# 1. Initialize a chat model
llm = ChatOpenAI(model="gpt-4o")

# 2. Send a message
response = llm.invoke([
    HumanMessage(content="What is LangChain in one sentence?")
])

# 3. Print the result
print(response.content)
```

**Key observation:** The `invoke()` method is the universal interface — every LangChain component supports it.

# Package Architecture Deep Dive

**Your Application**

**langchain** — Agents, Chains, Retrieval helpers

**langgraph** — Graph orchestration, state management

**langchain-core** — Runnables, Messages, Prompts, Tools, Parsers

**Provider Packages** — langchain-openai, langchain-anthropic, . . .

**Key Principle**

Dependencies flow **downward**: `langchain` depends on `langchain-core`, never the reverse.

# Core Abstractions Preview

| Abstraction | What It Does |
|---|---|
| **Runnable** | Universal protocol: anything with `invoke()`, `batch()`, `stream()` |
| **ChatModel** | Wraps an LLM provider; takes messages, returns messages |
| **PromptTemplate** | Formats variables into prompts |
| **OutputParser** | Converts raw LLM text into structured data |
| **Tool** | A function the LLM can decide to call |
| **Retriever** | Fetches relevant documents given a query |
| **Chain (LCEL)** | A pipeline of Runnables composed with \| |
| **Agent** | An LLM that plans and executes tool calls in a loop |

*Each of these is covered in depth in subsequent sections.*

[Docs: docs.langchain.com/oss/python/concepts]

# The Runnable Protocol

## The Universal Interface

Every LangChain component implements the **Runnable** protocol:

- `invoke(input)` — Process a single input
- `batch([inputs])` — Process multiple inputs in parallel
- `stream(input)` — Stream output token by token

Plus async variants: `ainvoke()`, `abatch()`, `astream()`

## Why This Matters

Because every component shares this interface, you can **compose** them freely with the pipe operator (`|`). Prompts, models, and parsers all plug in the same way.

# LCEL: LangChain Expression Language (Preview)

## Composing with the Pipe Operator

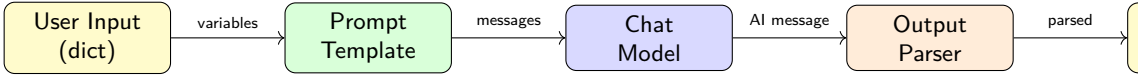
```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple terms."
)
model = ChatOpenAI(model="gpt-4o")
parser = StrOutputParser()

# Compose into a chain with |
chain = prompt | model | parser
result = chain.invoke({"topic": "quantum computing"})
```

*Full LCEL coverage in Section 03.*

# Data Flow: How a Request Moves Through LangChain




**Each Arrow is a Runnable Boundary**

At each step, the output of one Runnable becomes the input of the next. LangSmith tracing captures every boundary for debugging.

# LangChain vs Alternatives

| Feature | LangChain | LlamaIndex | Haystack | Raw SDK |
|---|---|---|---|---|
| Agent framework | ✓✓ | ✓ | ✓ | — |
| RAG pipeline | ✓✓ | ✓✓✓ | ✓✓ | Manual |
| Provider agnostic | ✓✓✓ | ✓✓ | ✓✓ | — |
| Graph orchestration | ✓✓✓ | — | — | — |
| Observability | ✓✓✓ | ✓ | ✓ | Manual |
| Community/ecosystem | ✓✓✓ | ✓✓ | ✓ | N/A |
| Learning curve | Medium | Medium | Medium | Low |

**Choosing LangChain**

LangChain excels at **agent orchestration**, **provider flexibility**, and **production observability**. LlamaIndex may be better for pure RAG-focused applications.

# What Changed: LangChain v0.x → v1.x

## Breaking Changes in v1.0+

- **LCEL is the standard** — Legacy chains (`LLMChain`, etc.) moved to `langchain-classic`
- **Provider packages split out** — e.g. `from langchain_openai import ChatOpenAI`
- **langchain-core extracted** — Interfaces live in `langchain-core`
- **Agents built on LangGraph** — `create_react_agent` returns a graph
- **Python 3.10+ required**

## Migration Tip

Start by updating imports to provider-specific packages, then replace legacy chains with LCEL pipelines.

# The Learning Path Ahead

**Foundation:** Models, Prompts, Parsing, LCEL (Sections 01–03)

↓

**Interaction:** Tools, Agents, Memory (Sections 04–06)

↓

**Retrieval:** Documents, Embeddings, RAG (Sections 07–09)

↓

**Observability:** Callbacks, Streaming, Tracing (Section 10)

↓

**Advanced:** LangGraph, Deep Agents (Sections 11–12)

↓

**Integration:** Full Project, Production (Sections 13–14)

# Summary

**Key Takeaways**

1. LangChain is a **framework for composing LLM-powered applications**, not an LLM itself
2. The ecosystem spans **four pillars**: langchain-core, langchain, LangGraph, LangSmith
3. Everything implements the **Runnable protocol**: `invoke()`, `batch()`, `stream()`
4. Components compose with the **pipe operator** (|) via LCEL
5. Provider packages are **separate**: install only what you need
6. v1.x is a **major redesign** — use LCEL, not legacy chains

# Cheat Sheet: Essential Imports

## Imports You'll Use Most Often

```python
# Core
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
# Models (pick your provider)
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
# Agents & Tools
from langchain.agents import create_react_agent
from langchain_core.tools import tool
# RAG
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
```

# Self-Check Questions

Test your understanding of this section:

1. What problem does LangChain solve that raw LLM SDKs do not?
2. Name the four pillars of the LangChain ecosystem and their roles.
3. What is the Runnable protocol, and why does it matter for composition?
4. Why are provider packages (e.g., `langchain-openai`) separate?
5. What is LCEL and what operator does it use?
6. How does LangChain v1.x differ from v0.x?
7. When might you choose **not** to use LangChain?

*Review any you're unsure about before proceeding to Section 01.*